

```
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler,PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge,LogisticRegression
from sklearn.neighbors import KNeighborsRegressor,KNeighborsClassifier
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import train_test_split
```

```
df = pd.read_csv("advanced.csv")
df.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6

```
X = df.drop("quality", axis=1)
y = df.quality
# y = (df["quality"] >= 7).astype(int) # this gives better result in Logistic Regression
```

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
```

```
lr = LogisticRegression()
lr.fit(X_train,y_train)
print(lr.score(X_test,y_test))
lrp = Pipeline([
    ("scaler", StandardScaler()),
    ("lr", LogisticRegression(max_iter=1000))
])

lrp.fit(X_train, y_train)
print(lrp.score(X_test, y_test))
```

```
0.553125
0.575
D:\Users\maamun\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
rd = Ridge()
rd.fit(X_train,y_train)
print(rd.score(X_test,y_test))
rdp = Pipeline([
    ("scaler", StandardScaler()),
    ("ridge", Ridge(alpha=1.0))
])

rdp.fit(X_train, y_train)
print(rdp.score(X_test, y_test))
```

```
0.39870643507758563
0.4031606598177524
```

```

rf = RandomForestRegressor(
    n_estimators=300,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42
)
rf.fit(X_train,y_train)
print(rf.score(X_test,y_test))
rfp = Pipeline([
    ("scaler", StandardScaler()),
    ("randomForest", RandomForestRegressor(
        n_estimators=300,
        max_depth=None,
        min_samples_split=2,
        min_samples_leaf=1,
        random_state=42
    ))
])

rfp.fit(X_train, y_train)
print(rfp.score(X_test, y_test))

```

```

0.5310802479282583
0.5307156545807452

```

```

knn = KNeighborsRegressor()
knn.fit(X_train,y_train)
print(knn.score(X_test,y_test))
knp = Pipeline([
    ("scaler", StandardScaler()),
    ("KNeighbors", KNeighborsRegressor())
])

knp.fit(X_train, y_train)
print(knp.score(X_test, y_test))

```

```

0.18592925775938085
0.3345507255039676

```

```

knnc = KNeighborsClassifier()
knnc.fit(X_train,y_train)
print(knnc.score(X_test,y_test))
kncp = Pipeline([
    ("scaler", StandardScaler()),
    ("KNeighbors", KNeighborsClassifier())
])

kncp.fit(X_train, y_train)
print(kncp.score(X_test, y_test))

```

```

0.45625
0.546875

```

Start coding or [generate](#) with AI.

```

lr_score = cross_val_score(lr,X,y,cv=5)
lrp_score = cross_val_score(lrp,X,y,cv=5)
rf_score = cross_val_score(rf,X,y,cv=5)
rfp_score = cross_val_score(rfp,X,y,cv=5)
rd_score = cross_val_score(rd,X,y,cv=5)
rdp_score = cross_val_score(rdp,X,y,cv=5)
knn_score = cross_val_score(knn,X,y,cv=5)
knp_score = cross_val_score(knp,X,y,cv=5)
print("Logestic Regression",lr_score, "average",lr_score.mean())
print("Logestic Regression with Pipeline",lrp_score, "average", lrp_score.mean())
print("Random Forest",rf_score, "average",rf_score.mean())
print("Random Forest with Pipeline",rfp_score, "average", rfp_score.mean())
print("Ridge",rd_score, "average",rd_score.mean())
print("Ridge with Pipeline",rdp_score, "average", rdp_score.mean())
print("KNearest Neighbour",knn_score, "average",knn_score.mean())
print("KNearest Neighbour with Pipeline",knp_score, "average", knnp_score.mean())
# print("logestic Regression",cross_val_score(lr,X,y,cv=5))

```

```
# print("logestic Regression with Pipeline",cross_val_score(lrp,X,y,cv=5))
# print("Ridge",cross_val_score(rd,X,y,cv=5))
# print("Ridge with Pipeline",cross_val_score(rdp,X,y,cv=5))
# print("KNeighbours ",cross_val_score(knn,X,y,cv=5))
# print("KNeighbours with Pipeline",cross_val_score(knnp,X,y,cv=5))
```

D:\Users\maamun\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

D:\Users\maamun\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

D:\Users\maamun\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

D:\Users\maamun\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

D:\Users\maamun\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
Logestic Regression [0.521875  0.559375  0.603125  0.5875    0.5862069] average 0.5716163793103448
Logestic Regression with Pipeline [0.51875  0.5375   0.63125   0.59375   0.63009404] average 0.5822688087774295
Random Forest [0.26048565 0.35467438 0.36633636 0.32197834 0.27528543] average 0.3157520341666474
Random Forest with Pipeline [0.25723325 0.35467506 0.36524213 0.32496276 0.27456403] average 0.3153354469949723
Ridge [0.13391596 0.32350868 0.35068596 0.36491063 0.27816901] average 0.2902380471700102
Ridge with Pipeline [0.13229651 0.31856638 0.34963649 0.36909878 0.28090939] average 0.29010151176504645
KNearest Neighbour [-0.01948843  0.01142882  0.03705885 -0.15589409 -0.11534666] average -0.04844830057356977
KNearest Neighbour with Pipeline [0.0453106  0.22767877 0.28863477 0.19285876 0.03766646] average 0.15842987045964957
```